Code Translation

4DV507/4DT902 – Code Transformation and Interpretation

Dr Suejb Memeti

suejb.memeti@lnu.se

Slides are available in Moodle

7 oktober 2026

Based on the original slides by Dr Jonas Lundberg

Agenda

- Introduction to Code Translation
- Python as the Target Language
- OFP-to-Python Translation
- Assignment 3: Overview

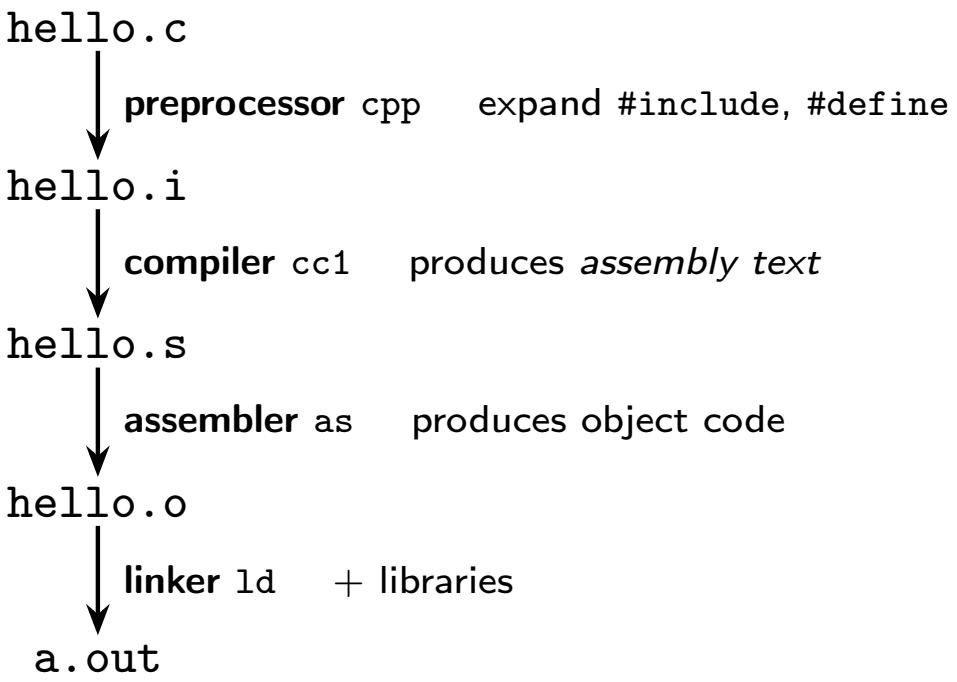
Code Translation Overview

- *Code translation* in a general sense = transformation from one code/language/representation to another
- Assembler: assembly language ⇒ machine code
- Compiler: source code ⇒ machine code (e.g., C, C++, Go), or source code ⇒ portable byte code (e.g., Java, Scala, C#)
- Interpreter: source code ⇒ intermediate representation executed statement-by-statement (e.g., Python, JavaScript)
- *Transpiler* (source-to-source compiler): source code1 ⇒ source code2 (e.g., Python 2 to Python 3, TypeScript to JavaScript, or C++ to WebAssembly)

Note: **the traditionally defined lines between compiled and interpreted languages are blurred nowadays—depending on the implementation, the same language could be interpreted, compiled into machine code, byte code, or transformed from byte code into machine code using a *Just-in-Time* (JIT) or *Ahead-of-Time* (AOT) compiler**

Where does the assembler fit?

The first two rows above look like they overlap. In the classic Unix pipeline they do not – each phase has its own program:



- So in GCC, the *compiler* really does stop at assembly, and a separate *assembler* finishes the job.
- **But this is a toolchain choice, not a law.**

Not every compiler emits assembly

- **Clang/LLVM** has an *integrated assembler* (since ≈2010): C ⇒ object code in one process. No `.s` file is ever written.
- **Go**'s compiler also emits object files directly.
- On macOS `gcc` *is* clang ⇒ try it yourself:

```
# one process, no assembler is ever invoked
$ clang -c t.c -###

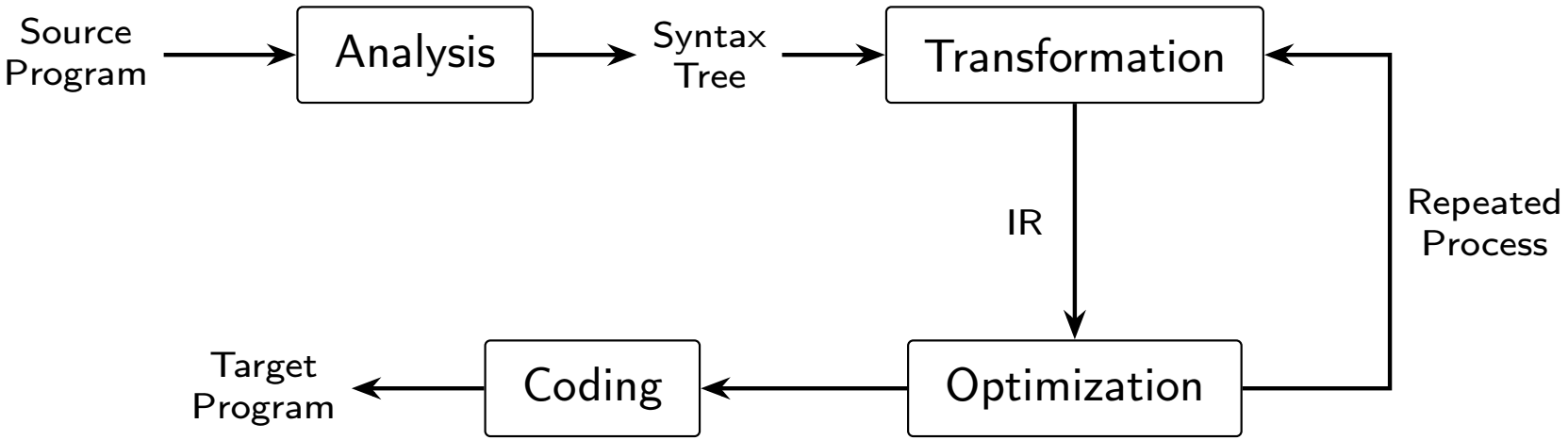
# the .s appears only because we asked for it
$ clang -S t.c

# now it DOES call /usr/bin/as
$ clang -fno-integrated-as -c t.c -###
```

- Same compiler, both behaviours, switched by a flag.

Conclusion: "compiler" describes *what comes out*, not how many programs it took to get there.

General Code Translation Pipeline

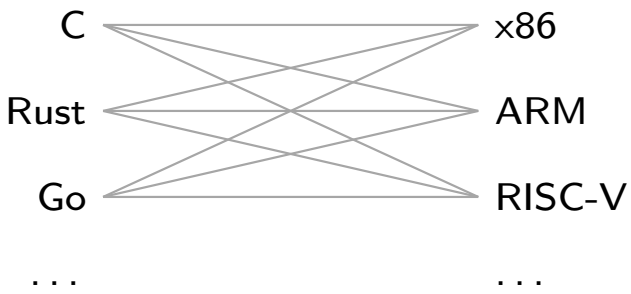


IR = Intermediate representation

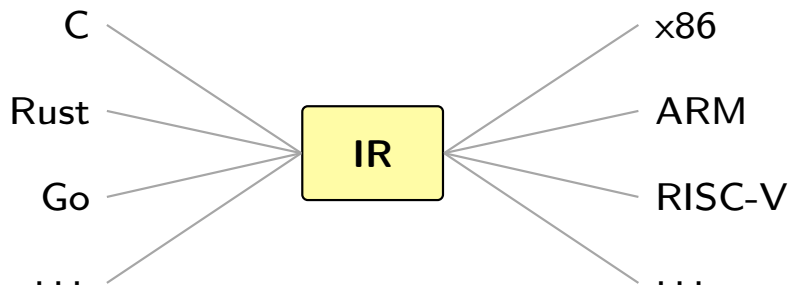
- IR is a machine- and source language-independent representation that can be seen as a connection between the front end and the back end of a compiler
- There are many types of IRs (e.g. basic block graphs, quadruple op-codes, SSA, byte code) used for different purposes (e.g. optimization, target code generation, stack machine execution)
- **OFP-to-Python translation approach ($\Rightarrow$ this lecture and Assignment 3)**:
  - Use the parse tree + symbol table
  - Skip any additional representations and optimizations
  - Generate Python source code as a string, and save it into a `.py` file

Why bother with an IR? The $M \times N$ problem

Suppose we support **5 source languages** and **5 target machines**.



without an IR
$M \times N$ translators
$5 \times 5 = 25$



front ends
one per language

back ends
one per machine

$M + N$ components $\quad 5 + 5 = 10$

The multiplication became an addition: $M \times N \Rightarrow M + N$.

What an IR actually buys you

- **Cheap growth.** A 6th language costs *one* front end and immediately runs on all 5 targets. Without an IR it costs 5 new translators. A new CPU costs one back end, and every language targets it at once.
- **Optimization written once.** Constant folding, dead code elimination, inlining, register allocation – implemented on the IR, so every language and every target gets them for free. This is the big one: optimization is most of a production compiler, and nobody wants to write it $M \times N$ times.
- **Analysis and verification.** Type checking, data-flow analysis and bytecode verification are easier on a small regular IR than on a big irregular source language.

Real examples: **LLVM IR** (Clang, Rust, Swift), **Java bytecode**, **.NET CIL**.

Java bytecode is this idea shipped as a product: `javac` is a front end, and the JVM on each platform is a back end. Nobody wrote a Java-to-ARM compiler – that is why "write once, run anywhere" works.

$\Rightarrow$ In **A4** you write a front end (OFP $\Rightarrow$ bytecode) and get every JVM platform for free.

Brief Introduction to Python

- Python is a popular high-level, general-purpose programming language that supports multiple paradigms (procedural, object-oriented, functional)
- We will use Python 3 for our purposes (Python 2 is deprecated)
- Standard implementation (*CPython*): an interactive interpreter (input the source code directly) + bytecode generation when executing `.py` source code files (`.pyc` bytecode files are created for later executions)
- You must have a working Python environment to test all generated `.py` programs produced by your implementation
- If you do not have Python 3 installed, get the installer from `https://www.python.org/downloads/` (e.g., version 3.8) or some other location/package, or use an online interpreter such as `https://repl.it/languages/python3`

Brief Introduction to Python (cont.)

- Python supports garbage collection, dynamic strong typing, and implicit type expressions
  - No concerns about explicit variable declaration before usage
  - . . . or explicit function return type declaration
  - But adding an integer to a boolean is still an error!
- Instead of curly braces, Python source code relies heavily on proper *indentation* $\Rightarrow$ be careful!
- We will only focus on a small subset of features of Python and disregard support for classes, packages, imported libraries, etc.
- For our purposes, a valid `.py` file consists of:
  1. zero or more function declarations, and
  2. zero or more statements at the end of the script file (executed at the program entry point) $\Rightarrow$ the main function

Note: if necessary, the complete Python language reference is available from `https://docs.python.org/3.8/reference/`

Example Python Program

```
def test_function(array, index): # no explicit type annotations
    if len(array) < index: # note the indentation!!!
        print("Index error!") # print and end the line with \n
        return

    i = 0 # no explicit variable declarations
    while i < index + 1:
        print(array[i], end=" ") # no line break
        i = i + 1
    print()

# Program entry point
test_array = [100, 200, 300]          # List creation
test_function(test_array, 2)
test_function(test_array, 100)        # Index error
```

OFP vs Python

while.ofp

```
void main() {
    int n = 10;
    int res = sumUpTo(n);
    println(res);    # 55
}

# Computes 1+2+3+...+n)
int sumUpTo(int n) {
    int i = 1;
    int sum = 0;
    while (i < n+1) {
        sum = sum + i;
        i = i + 1;
    }
    return sum;
}
```

while.py

```
def sumUpTo(n):
    i = 1
    ofp_sum = 0
    while i < n + 1:
        ofp_sum = ofp_sum + i
        i = i + 1
    return ofp_sum

#
# Program entry point - main
#
n = 10
res = sumUpTo(n)
print(res)
```

Notice

- the contents of the OFP main function were translated into inline code at the end of the Python program;
- the function `sum` and variable `len` in OFP had to be renamed to avoid conflict with a Python built-in function;

OFP-to-Python Translation

The task of translating OFP programs to Python source code files is rather straightforward:

1. Parse the input `.ofp` file and produce a parse tree ⇒ Assignment 1
2. Collect the information about the symbols ⇒ Assignment 2
3. Use the parse tree + symbol table to generate Python source code:
   - Create a new class, `PythonCodeGenerator`, as a **visitor**
   - For each function and statement in OFP, produce the corresponding Python code as a *text string* as you traverse the parse tree
   - Store the complete Python code text in an external `.py` file with the same name as the OFP file

Remember: our goal is to produce Python code that **behaves** as input OFP code, even if it looks somewhat different ⇒ the *observable behavior* (e.g., printed computation results) should be the same* !

- Minor differences in the output (e.g., OFP `true` and `false` literals printed as `True` and `False` by Python runtime) are OK if the semantics are preserved

Python Code Generation Concerns

OFP is a very simple language, so we should achieve *almost* 1:1 mapping between our parse tree nodes and output Python code, but...

- **Issue 1:** how can we generate Python code as a text string?
- **Issue 2:** how do we address the indentation requirements?
- **Issue 3:** what should we do with variable declarations?
- **Issue 4:** what should we do with the `void main()` function?
- **Issue 5:** does OFP allow something that is prohibited in Python?
- **Issue 6:** a few particular OFP constructs

Issue 1: Code String Generation

- Depending on the parse tree node that we are currently visiting, we should produce the corresponding Python code and return it as a string
- Example:

```
@Override
public String visitBool(OFPParser.BoolContext ctx) {
  String b = ctx.getChild(0).getText();
  if (b.equals("true"))
    return "True";
  else
    return "False";
}
```

Basic idea

- Each node returns a string representing a code fragment
- Parent nodes form a new string using the child fragments
- The root node puts everything (all function text fragments) to a complete program

Issue 1: Code String Generation (cont.)

- A lot of string construction required ⇒ use the *StringBuilder* class for a temporary string buffer
- Example: Array initialization is handled using Python lists

```
@Override // '{' expr (',' expr)* '}'
public String visitInitArrayExpr(
                        OFPParser.InitArrayExprContext ctx) {
  StringBuilder buf = new StringBuilder();
  buf.append( visit(ctx.getChild(1)) );    // First expression
  for (int i=3; i<ctx.getChildCount();i+=2) { // Remaining expressions
    buf.append(", ").append( visit(ctx.getChild(i)) );
  }
  return "[" + buf.toString() + "]";   // A new Python list
}
```

Issue 2: Indentation

- Python code must be properly indented ⇒ keep track of the current indentation level, increase/decrease it accordingly, and produce **four** space characters per each indentation level
- Indentation should typically change for blocks of OFP statements, e.g., when entering the function body or a then/else block
- The contents of the main function will get a special treatment (see below)
- Also make sure to add **new line** characters when necessary ⇒ typically, after each OFP statement and after some parts of complex statements such as `if` and `while`

Issue 2: Indentation (cont.)

- You might want to cache the strings of space characters for indentation purposes in your visitor:

```
private int depth = 0; // The current depth/indentation level
private HashMap<Integer,String> indentCache =
                                new HashMap<Integer,String>();

private String indent(int indentLevel) {
  String ind = indentCache.get(indentLevel);
  if (ind == null) {
    ind = "";
    for (int i=0; i < indentLevel; i++)
        ind += "    ";
    indentCache.put(indentLevel, ind);
  }
  return ind;
}
```

Issue 2: Indentation (concl.)

- Example for visiting a block of statements:

```
@Override
public String visitBlock(OFPParser.BlockContext ctx) {
  currentScope = scopes.get(ctx);
  depth++;

  StringBuilder buf = new StringBuilder();
  for (int i=1; i<ctx.getChildCount()-1;i++) {    // Visit statements
      String stmt = visit(ctx.getChild(i));
    buf.append(stmt);
  }

  if (buf.length() == 0) // handle empty blocks
    buf.append(indent(depth)).append("pass").append("\n");

  depth--;
  currentScope = currentScope.getEnclosingScope();
  return buf.toString();
}
```

Issue 3: Variable Declarations

- We can ignore variable declarations unless the variable is initialized/assigned some value
- Example:

```
@Override // type ID ('=' expr)?
public String visitVarDecl(OFPParser.VarDeclContext ctx) {
  if (ctx.getChildCount() > 2) {
    String id = ctx.getChild(1).getText();
    id = safeID(id);    // See issue 5
    String expr = visit(ctx.getChild(3));
    return id + " = " +expr ;
  }
  else // ignore pure variable declarations
    return "pass";
}
```

Issue 4: The Main Function

- When visiting the parse tree root, skip the visit to the main function, and save it for last
- The body of main should then be simply appended to the output buffer:

```
@Override // 'void' 'main' '(' ')' funcBody;
public String visitMain(OFPParser.MainContext ctx) {
  currentScope = scopes.get(ctx);
  String start = "#\n# Program entry point - main\n#\n";

  depth = 0;    // No indentation expected here ==> reset depth
  String body = visit(ctx.getChild(4)); // String for main body

  currentScope = currentScope.getEnclosingScope();
  return start + body;
}
```

Issue 5: Illegal Constructs

- What if the source OFP program uses variable or function names such as `len` or `def` $\Rightarrow$ reserved keywords in Python!
- Of course, we would still be able to produce a `.py` file, but it would be invalid
- Solution: check the OFP identifiers (function, parameter, and variable names) against a list/set of reserved names, and modify the identifiers, if necessary
- We should start with a list of Python keywords, builtin functions, and some standard classes such as exceptions
- The list mentioned in the next slide was obtained for Python 3.8. A snippet of Java code with this list will be provided with Assignment 3 instructions!

Issue 5: Illegal Constructs (cont.)

- We use `java.util.HashSet` and `java.util.Arrays` here:

```
private static HashSet<String> reservedIds =
    new HashSet<String>(Arrays.asList("def", "len", ...
                    // The complete list will be provided
                    // in Assignment 3

private static String safeID(String id) {
  if (reservedIds.contains(id))
        // Underscore is illegal in OFP identifiers
        // so no chance of collision with other IDs
    return "ofp_" + id;
  else
    return id;
}
```

- Use `safeID()` to produce safe identifiers in your output code ⇒ but see the next issue first!

Issue 6: Particular OFP Constructs

- OFP `println` statement ⇒ use the Python in-built `print()` function
- OFP `print` statement ⇒ `print(..., end='')` in Python
- OFP `.length` operation for arrays and strings ⇒ use the Python in-built `len()` function
- OFP array initialization ⇒ see the next slide
- . . . ⇒ it's your task to come up with the implementation for the rest!

Issue 6: Particular OFP Constructs (cont.)

- OFP new array initialization ⇒ create empty Python lists

```
@Override. // 'new' (Int|Float|Char) '[' expr ']'
public String visitNewArrayExpr(
                    OFPParser.NewArrayExprContext ctx) {
  String type = ctx.getChild(1).getText();
  String expr = visit(ctx.getChild(3));
  if (type.equals("int")) {
    return "[0]*"+"("+ expr+")";       // "[0]*N" used in Python to produce
  }                                    // a list of N duplicate values
  else if (type.equals("float") {
    return "[0.0]*"+"("+ expr+")";
  } else {
    return ['']*"+"("+ expr+")";
  } }
```

- For OFP array literals such as `{1,2,3}`, simply produce the corresponding Python lists: `[1,2,3]`

PythonCodeGenerator Visitor Sketch

```
public class PythonCodeGenerator extends OFPBaseVisitor<String> {
  private ParseTreeProperty<Scope> scopes;
  private Scope currentScope;

  // see Issue 2
  private int depth = 0;
  private HashMap<Integer,String> indentCache = ...
  private String indent(int indentLevel) { ... }

  // see Issue 5
  private static HashSet<String> reservedIds = ...
  private static String safeID(String id) { ... }

  public PythonCodeGenerator(ParseTreeProperty<Scope> scopes) {
    this.scopes = scopes;
  }
```

PythonCodeGenerator Visitor Sketch (cont.)

```
@Override
public String visitProgram(OFPParser.ProgramContext ctx) {
  currentScope = scopes.get(ctx);
  StringBuilder buf = new StringBuilder();   // For generated Python code

  // main function must be generated last!
  ParserRuleContext main = null;

  for (int i=0; i<ctx.getChildCount();i++) {
    ParserRuleContext p = (ParserRuleContext) ctx.getChild(i);
    String fName = p.getChild(1).getText();
    if (fName.equals("main"))
      main = p;      // defer visit until after the rest of the functions
    else
      buf.append(visit(p));    // Visit non-main functions
  }

  buf.append(visit(main));

  return buf.toString();     // Return entire code as a string
}
```

Assignment 3: Overview

- **Task:** extend your implementation to translate the input OFP programs (`X.ofp` files) to executable Python programs (`X.py` files)
  - You can assume that input programs are valid OFP programs (but we will test for identifiers reserved in Python and other issues discussed in this lecture!)
  - Your type checking implementation (Assignment 2) should not affect your grade for Assignment 3
  - Make sure that valid OFP programs are accepted by your parser + symbol table + type checker, though ⇒ if we cannot test Assignment 3 tasks because of Assignment 2 issues, you might get in trouble!
- **Deadline: see Moodle**
  ⇒ Start early, and you should be able to complete it on time!
- Detailed Assignment 3 instructions are available in Moodle